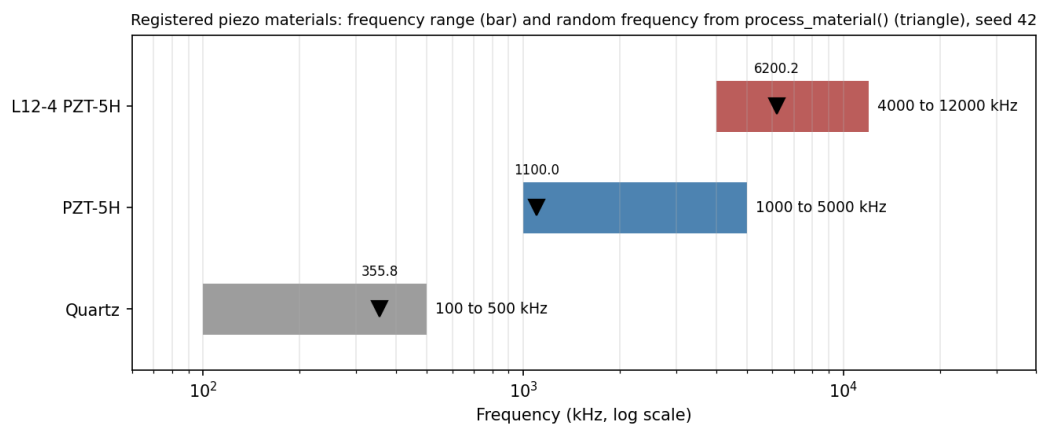


Team 1: Transducer

A registry of piezoelectric materials and their frequency ranges

NITK-UsoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- Team 1 stores piezoelectric materials (such as Quartz and PZT-5H) with a checked frequency range.
- A registry lets the rest of the program add a material and look it up by name.
- **process_material()** returns a random frequency inside a material's range.
- One command: `python3 run_team1.py` (about 1 second).



Registered materials and the random frequency returned for each (real run, seed 42).

Contents

1 Overview	4
Files in the folder	4
2 Theory	5
The piezoelectric effect	5
Common materials	5
Frequency range and bandwidth	5
Units used by the code	6
Why validation matters	6
The registry pattern and custom exceptions	6
Random frequency	6
3 Code explanation	7
exceptions.py	7
piezo.py	7
registry.py	7
new_piezo.py	8
add_attributes.py	8
process.py	8
main.py	8
Full listings of the team's files	8
4 How run_team1.py uses the team's own code	12
Where run_team1.py uses your code	12
What run_team1.py adds on top (written for this demo, not the team's code)	13
run_team1.py step by step	14
Full listing of run_team1.py	14
5 Input	19
Demo 1: values written inside main.py	19
Demo 2: the project probe	19
Demo 3: wrong inputs	19
6 Output	20
Registry contents	20
Figure 1: material frequency ranges	20
Figure 2: random frequencies for the L12-4 probe	21
Console output	21
Console screenshot	23
7 How to run	24

Troubleshooting	24
---------------------------	----

8 Check yourself	25
-------------------------	-----------

1 Overview

What Team 1 does and what is in the folder

An ultrasound probe contains small pieces of **piezoelectric** material. They turn an electric voltage into a sound vibration and a returning echo back into a voltage. Team 1 wrote the part of the simulator that **describes these materials**: each material has a name and a frequency range (minimum and maximum frequency). The code checks that each range makes sense, keeps all materials in one **registry**, and can return a random frequency inside a material's range.

Place in the pipeline: Team 1 is the first block (the transducer). It describes the probe material and band that the later teams (transmit beamforming, propagation, receive) rely on. The positions of the 128 array elements are computed elsewhere in the project (Team 3's function), not in Team 1's code.

Files in the folder

File / folder	What it is
README.md	Short run instructions
requirements.txt	Python package needed (matplotlib, only for the figures)
run_team1.py	Runner added for this demo: calls the team's code and saves every output
code/piezo.py	Team code: the Piezo class with range validation
code/exceptions.py	Team code: custom error classes
code/registry.py	Team code: PiezoRegistry (a dictionary of materials) and the shared registry
code/new_piezo.py	Team code: newPiezoMaterial(), adds an empty material
code/add_attributes.py	Team code: addNewAttributes(), sets or updates a range
code/process.py	Team code: process_material(), register and/or return a random frequency
code/main.py	Team code: the team's own demo
input/team1_inputs.json	Sample inputs: random seed, probe values, wrong inputs
output/	Figures, CSV table, console log and console screenshot
Team_1_Transducer_Explanation.pdf	This document

Original vs added

Everything in **code/** is the team's original work, copied byte-for-byte without change.

run_team1.py, **input/**, **output/**, **README** and this PDF were added so the part can be run and shown on its own.

2 Theory

The ideas behind the code, in simple words

The piezoelectric effect

Some crystals and ceramics produce a voltage when they are squeezed, and change shape when a voltage is applied. This is the **piezoelectric effect**. It works both ways:

- **Transmit:** an electric pulse makes the element vibrate, and it sends a sound wave into the body.
- **Receive:** the echo coming back shakes the element, and it produces a small voltage.
- The **same element** is used for both, first as a loudspeaker and then as a microphone.

A thin plate vibrates most strongly at a resonance frequency set by its thickness d and the speed of sound c inside the material (half-wavelength plate):

$$f_0 \approx \frac{c_{\text{material}}}{2d}$$

Thinner element gives a higher frequency.

Common materials

Material	What it is	Use
Quartz	Natural single crystal. Very stable, but a weak piezo effect.	Clocks, oscillators, early ultrasound. Team demo range 100 to 500 kHz.
PZT-5H	Lead zirconate titanate ceramic, 'soft' type. Very strong piezo effect, easy to shape.	Most medical imaging probes. The project's L12-4 probe uses PZT-5H, 4 to 12 MHz.

PZT is used in medical probes because it converts energy much more efficiently than quartz (strong coupling), so weak echoes still give a usable signal, and it can be cut into many small elements for an array (the L12-4 has 128).

Frequency range and bandwidth

A probe does not work at one single frequency. It works over a **band** from a minimum to a maximum frequency. Useful numbers:

$$B = f_{\max} - f_{\min}, \quad f_c = \frac{f_{\min} + f_{\max}}{2}, \quad FBW = \frac{B}{f_c}$$

For the L12-4 probe: $f_{\min} = 4$ MHz, $f_{\max} = 12$ MHz, so $B = 8$ MHz, $f_c = 8$ MHz and the fractional bandwidth is 100 % (these values are printed by the run). A wide band means a short pulse and good depth (axial) resolution. Higher frequency gives finer detail but less depth, because tissue absorbs high frequencies more.

$$\lambda = \frac{c}{f}, \quad \lambda_{8\text{ MHz}} = \frac{1540 \text{ m/s}}{8 \times 10^6 \text{ Hz}} \approx 0.19 \text{ mm}$$

Wavelength in soft tissue at the centre frequency.

Units used by the code

The team's `main.py` prints frequencies in **kHz**, so this demo uses kHz everywhere: 4 to 12 MHz is written as 4000 to 12000 kHz. The code itself does not store a unit; it only stores two numbers.

Why validation matters

- A range is two numbers: (min, max). Anything else is rejected.
- Frequencies cannot be negative.
- min must not be larger than max, otherwise 'a random frequency in the range' has no meaning.
- Values must be numbers; text like 'four MHz' is rejected.
- Catching a bad value early, with a clear message, is much better than a wrong image later in the pipeline.

The registry pattern and custom exceptions

A **registry** is one shared place where objects are stored by name. Here it is a Python dictionary: the key is the material name in lower case, the value is the Piezo object. Adding is `dict[key] = obj`; looking up is `dict.get(key)`. Because the key is lower case, 'PZT-5H' and 'pzt-5h' find the same material.

Custom exceptions give errors a clear name. `PiezoValidationError` means 'bad range'; `PiezoNotFoundError` means 'no such material'. Both come from `PiezoError`, so one except `PiezoError` can catch all of them. They also inherit from `ValueError` / `KeyError`, so normal Python error handling still works.

Random frequency

`process_material()` uses `random.uniform(min, max)`: every value between min and max is equally likely (a uniform distribution). The team code sets no seed, so the numbers change every run. The runner calls `random.seed(42)` first so the output is repeatable.

3 Code explanation

Every team file in code/, section by section

The seven files import each other by plain name (for example from `registry import registry`), so they must sit in the same folder. The chain is: `exceptions.py` → `piezo.py` → `registry.py` → `new_piezo.py` / `add_attributes.py` → `process.py` → `main.py`.

exceptions.py

- **Lines 1-4:** docstring.
- **PiezoError(Exception):** the base class for every Team 1 error. It adds nothing, it only gives a family name.
- **PiezoValidationError(PiezoError, ValueError):** raised when a range is wrong. It is both a `PiezoError` and a `ValueError` (multiple inheritance).
- **PiezoNotFoundError(PiezoError, KeyError):** raised when a name is not in the registry. It is both a `PiezoError` and a `KeyError`. Because it is a `KeyError`, Python prints its message with quotes around it (you can see this in the console output).

piezo.py

- **import:** takes `PiezoValidationError` from `exceptions.py`.
- **class Piezo, __init__(name, freq_range=(0.0, 0.0)):** stores the name, then sets `self.freq_range`. This goes through the property setter below, so even the first value is checked.
- **@property freq_range (getter):** returns the stored private value `self._freq_range`.
- **@freq_range.setter:** runs every time someone writes `obj.freq_range = ...`. Four checks in order:
 1. value must be a tuple or list of length 2, else error 'must be a tuple or list of 2 numbers';
 2. both items are converted with `float()`; if this fails the error says 'must be numeric' and shows Python's detail;
 3. if either is negative: 'Frequencies must be non-negative';
 4. if `min > max`: 'Minimum frequency ... cannot exceed maximum frequency ...'.
- **Last line:** only if all checks pass, the range is saved as a tuple of two floats. A list input is therefore stored as a tuple, and integers become floats.

registry.py

- **class PiezoRegistry, __init__:** creates an empty dictionary `_materials` (name → `Piezo`).
- **add_material(name, freq_range):** builds a `Piezo` (so the range is checked), stores it under `name.lower()`, and returns it. Adding the same name again replaces the old entry.
- **get_material(name):** looks up `name.lower()`; if nothing is found raises `PiezoNotFoundError`.
- **update_attributes(name, freq_range):** finds the material (error if missing) and sets a new range (checked by the setter).
- **list_materials():** returns the original names (with their capital letters) of all materials, in the order they were added.
- **clear():** empties the dictionary.
- **registry = PiezoRegistry():** one shared object created when the module is first imported. Every other file imports this same object, so they all see the same materials.

new_piezo.py

- **newPiezoMaterial(name):** adds a material with the placeholder range (0.0, 0.0) and returns the Piezo object. (It imports Piezo only for the return type hint.)

add_attributes.py

- **addNewAttributes(name, freq_range):** tries `registry.update_attributes()`. If the material does not exist (`PiezoNotFoundError`), it adds it instead. So it means 'update or create'. A `PiezoValidationError` is not caught here, so it goes up to the caller.

process.py

- **process_material(name, freq_range=(0.0, 0.0), create_new=True):** the main function.
- If **create_new** is True: first `newPiezoMaterial(name)` adds the material with (0.0, 0.0), then `addNewAttributes(name, freq_range)` sets the real range.
- Then it always looks the material up, reads (min, max) and returns `random.uniform(min, max)`.
- With **create_new=False** it only looks up an existing material; the `freq_range` argument is ignored.
- **Side effect to know:** if the range is invalid, the error is raised in step 2, but step 1 already added the material with (0.0, 0.0). So the name stays in the registry (the runner shows this in Demo 3b).

main.py

- **Section 1:** registers Quartz (100-500) and PZT-5H (1000-5000) and prints the random frequency of each, in kHz, and the list of materials.
- **Section 2:** calls `process_material('Quartz', create_new=False)` to look Quartz up again and prints a new random frequency.
- **Section 3:** tries 'BadRange' with (500.0, 1000.0) inside try/except `PiezoValidationError`. This range is actually valid, so no error is raised, nothing is printed, and BadRange is added. Then it looks up 'UnknownMat', which raises `PiezoNotFoundError`, and the message is printed.
- **if __name__ == '__main__':** runs `main()` only when the file is run directly, not when it is imported (the runner imports it and calls `main()`).

Full listings of the team's files

code/exceptions.py

```

1  """
2  exceptions.py
3  Custom exception hierarchy for Piezo operations.
4  """
5
6  class PiezoError(Exception):
7      """Base exception for all Piezo-related errors."""
8      pass
9
10
11 class PiezoValidationError(PiezoError, ValueError):
12     """Raised when frequency ranges or attributes fail validation checks."""
13     pass
14
15
16 class PiezoNotFoundError(PiezoError, KeyError):
17     """Raised when a requested material is not found in the registry."""
18     pass

```

code/piezo.py

```

1  """
2  piezo.py
3  Piezo data class with property setter validations.
4  """
5
6  from exceptions import PiezoValidationError
7
8
9  class Piezo:
10     """Represents a piezoelectric material with validated frequency range properties."""
11
12     def __init__(self, name: str, freq_range: tuple[float, float] = (0.0, 0.0)):
13         self.name = name
14         self.freq_range = freq_range
15
16     @property
17     def freq_range(self) -> tuple[float, float]:
18         return self._freq_range
19
20     @freq_range.setter
21     def freq_range(self, value: tuple[float, float]) -> None:
22         """Validates that frequency range consists of two non-negative numbers where min <= max."""
23         if not isinstance(value, (tuple, list)) or len(value) != 2:
24             raise PiezoValidationError(
25                 "freq_range must be a tuple or list of 2 numbers (min_freq, max_freq)."
26             )
27
28         try:
29             min_freq, max_freq = float(value[0]), float(value[1])
30         except (ValueError, TypeError) as e:
31             raise PiezoValidationError(f"Frequency values must be numeric. Details: {e}")
32
33         if min_freq < 0 or max_freq < 0:
34             raise PiezoValidationError(
35                 f"Frequencies must be non-negative. Got: ({min_freq}, {max_freq})"
36             )
37
38         if min_freq > max_freq:
39             raise PiezoValidationError(
40                 f"Minimum frequency ({min_freq}) cannot exceed maximum frequency ({max_freq})."
41             )
42
43         self._freq_range = (min_freq, max_freq)

```

code/registry.py

```

1  """
2  registry.py
3  Registry class for managing multiple material objects and global registry instance.
4  """
5
6  from piezo import Piezo
7  from exceptions import PiezoNotFoundError
8
9
10 class PiezoRegistry:
11     """Dynamic registry to store, look up, and manage multiple Piezo materials."""
12
13     def __init__(self):
14         self._materials: dict[str, Piezo] = {}
15
16     def add_material(self, name: str, freq_range: tuple[float, float] = (0.0, 0.0)) -> Piezo:
17         """Creates a material and adds it to the registry."""
18         material_obj = Piezo(name=name, freq_range=freq_range)
19         self._materials[name.lower()] = material_obj
20         return material_obj
21
22     def get_material(self, name: str) -> Piezo:
23         """Looks up a material by name (case-insensitive). Raises PiezoNotFoundError if missing."""
24         material_obj = self._materials.get(name.lower())
25         if not material_obj:
26             raise PiezoNotFoundError(f"Material '{name}' was not found in the registry.")
27         return material_obj
28

```

```

29     def update_attributes(self, name: str, freq_range: tuple[float, float]) -> None:
30         """Updates frequency range attributes of an existing material."""
31         material_obj = self.get_material(name)
32         material_obj.freq_range = freq_range
33
34     def list_materials(self) -> list[str]:
35         """Returns a list of all registered material names."""
36         return [mat.name for mat in self._materials.values()]
37
38     def clear(self) -> None:
39         """Clears all stored materials from the registry."""
40         self._materials.clear()
41
42
43 # Shared global instance used across modules
44 registry = PiezoRegistry()

```

code/new_piezo.py

```

1  """
2  new_piezo.py
3  Function to instantiate and register a new Piezo material.
4  """
5
6  from piezo import Piezo
7  from registry import registry
8
9
10 def newPiezoMaterial(name_of_material: str) -> Piezo:
11     """Creates a new Piezo object and registers it."""
12     return registry.add_material(name=name_of_material, freq_range=(0.0, 0.0))

```

code/add_attributes.py

```

1  """
2  add_attributes.py
3  Function to add/update material attributes in the registry.
4  """
5
6  from registry import registry
7  from exceptions import PiezoNotFoundError
8
9
10 def addNewAttributes(name_of_material: str, freq_range: tuple[float, float]) -> None:
11     """Updates property values for a named material in the registry."""
12     try:
13         registry.update_attributes(name_of_material, freq_range)
14     except PiezoNotFoundError:
15         registry.add_material(name_of_material, freq_range)

```

code/process.py

```

1  """
2  process.py
3  Function for executing the main operational block logic.
4  """
5
6  import random
7  from registry import registry
8  from new_piezo import newPiezoMaterial
9  from add_attributes import addNewAttributes
10
11
12 def process_material(
13     name_of_material: str,
14     freq_range: tuple[float, float] = (0.0, 0.0),
15     create_new: bool = True,
16 ) -> float:
17     """
18     Main processing function to create/update material definitions
19     and extract a random float frequency within specified boundaries.
20     """
21     if create_new:

```

```

22     newPiezoMaterial(name_of_material)
23     addNewAttributes(name_of_material, freq_range)
24
25     piezo_obj = registry.get_material(name_of_material)
26     min_freq, max_freq = piezo_obj.freq_range
27
28     return random.uniform(min_freq, max_freq)

```

code/main.py

```

1  """
2  main.py
3  Main entry point script demonstrating end-to-end usage across modular imports.
4  """
5
6  from registry import registry
7  from process import process_material
8  from exceptions import PiezoValidationError, PiezoNotFoundError
9
10
11 def main():
12     print("--- 1. Registering Materials ---")
13     q_freq = process_material("Quartz", (100.0, 500.0), create_new=True)
14     pzt_freq = process_material("PZT-5H", (1000.0, 5000.0), create_new=True)
15
16     print(f"Quartz generated frequency : {q_freq:.2f} kHz")
17     print(f"PZT-5H generated frequency: {pzt_freq:.2f} kHz")
18     print(f"Registered materials      : {registry.list_materials()}\n")
19
20     print("--- 2. Retrieving Existing Material ---")
21     fetched_freq = process_material("Quartz", create_new=False)
22     print(f"Retrieved Quartz frequency : {fetched_freq:.2f} kHz\n")
23
24     print("--- 3. Testing Exception Handling ---")
25     try:
26         process_material("BadRange", (500.0, 1000.0), create_new=True)
27     except PiezoValidationError as e:
28         print(f"Validation failure caught: {e}")
29
30     try:
31         process_material("UnknownMat", create_new=False)
32     except PiezoNotFoundError as e:
33         print(f"Lookup failure caught      : {e}")
34
35
36 if __name__ == "__main__":
37     main()

```

4 How run_team1.py uses the team's own code

Team logic from code/ (unchanged) and the demo wrapper around it

All the logic of this part comes from the team's existing code in **code/**: the Piezo class and its range checks, the error classes, the registry, newPiezoMaterial(), addNewAttributes(), process_material() and main.py. These files are used **unchanged**. They are not rewritten, not copied into the runner and not edited. **run_team1.py** is a demo wrapper: it imports the team's modules, calls them with the demo inputs, and prints and saves what they return.

Where run_team1.py uses your code

run_team1.py line(s)	Team file / function used	What it does there
94	code/main.py (module)	Imports the team's demo script as team_main.
95	code/registry.py: the shared registry object (registry.py line 44)	Imports the same registry object that the team's functions use.
96	code/process.py: process_material()	Imports the main team function.
97	code/piezo.py: class Piezo	Imported for Demo 3a.
98	code/exceptions.py: PiezoError, PiezoValidationError, PiezoNotFoundError	Imported to catch and check the team's errors.
108	main.py: main()	Demo 1: runs the team's own demo exactly as written.
110-111	registry.list_materials(), registry.get_material(), Piezo.freq_range	Prints the registry after Demo 1.
120	registry.clear()	Empties the registry before the repeat check.
121-122	process_material(..., create_new=True)	Registers Quartz and PZT-5H again to show the numbers repeat.
136	process_material(..., create_new=True)	Demo 2: registers 'L12-4 PZT-5H' with 4000 to 12000 kHz and gets one random frequency.
140	registry.get_material(); Piezo.freq_range is read at lines 141-143	Looks the probe up with a lower-case name (the team's lookup is case-insensitive).
149	process_material(..., create_new=False)	Demo 2: 2000 look-up-only calls.
155	registry.list_materials()	Prints the registered names.
159-162	registry.list_materials(), registry.get_material(), Piezo.freq_range	Collects the registry contents for the CSV and Figure 1.
171, 173	Piezo(name, freq_range): the freq_range setter in piezo.py; PiezoValidationError	Demo 3a: builds Piezo objects with five wrong ranges and catches the team's error.
180-181	process_material(..., create_new=True); PiezoValidationError	Demo 3b: the same kind of wrong range through the main function.
183	registry.get_material()	Shows that 'Swapped' stays in the registry with (0.0, 0.0).
189-190	registry.get_material(); PiezoNotFoundError	Demo 3c: looks up a name that was never registered.
194-197	exceptions.py: the three error classes	Demo 3d: prints the error family (PiezoError / ValueError / KeyError).

Inside the team's code, `process_material()` calls `newPiezoMaterial()` (`new_piezo.py`) and `addNewAttributes()` (`add_attributes.py`) at `process.py` lines 22-23. So those two team functions also run every time the runner calls `process_material()` with `create_new=True`.

What run_team1.py adds on top (written for this demo, not the team's code)

run_team1.py line(s)	Added for this demo	Why
25-29	Folder paths; creates output/	Finds code/, input/ and output/.
33-34	Puts code/ on the import path; no .pyc files	The team's files import each other by plain name; code/ stays clean.
36-39	matplotlib with the 'Agg' backend	Figures are saved to PNG files instead of being shown in a window. (The team's code draws no figures.)
42-54, 258-265	Tee class: keeps a copy of everything printed	Saved as output/console_log.txt.
57-60	banner()	Prints the DEMO headings.
63-81, 266	console_to_png()	Draws the console text as output/console_screenshot.png.
85, 254	Timer	Prints the run time.
86-87	Reads input/team1_inputs.json	All demo inputs come from this file.
91, 119	random.seed(42) (fixed random seed)	The team's <code>process_material()</code> uses <code>random.uniform()</code> with no seed; fixing it makes every run print the same numbers.
100-104	Header lines	Says what is run, the input file, the seed and the unit.
109-115	Registry print-out and a note	Explains that main.py's 'BadRange' range (500, 1000) is valid.
117-125	Repeat check	Clears the registry, resets the seed and repeats two registrations.
129-134	Probe values for Demo 2	Reads and prints the L12-4 values; converts kHz to MHz for printing.
142-146	Own calculations: $\text{bandwidth} = \text{max} - \text{min}$, $\text{middle} = (\text{min} + \text{max}) / 2$, $\text{fractional bandwidth} = \text{bandwidth} / \text{middle} \times 100 \%$	Demo additions. These formulas are not in the team's code.
150-154	Own statistics of the 2000 values: smallest, largest, average, number inside the range	Demo addition, to check that every value is inside the range.
158-162	Table of registry rows	Used for the CSV and Figure 1.
166-176	Loop over the five wrong inputs from the JSON file, with try/except and print	Demo 3a test cases.
184-185	Explanatory note	Explains why 'Swapped' stays in the registry.
200-207	output/registry_contents.csv, with an extra bandwidth_khz column (hi - lo, line 206)	Demo addition: the team's code does not write files.
209-230	Figure 1: bar chart of each material's range with the returned frequency	Demo addition.
232-250	Figure 2: histogram of the 2000 values, with lines at 4, 8 and 12 MHz	Demo addition.

To see it yourself

To see it yourself: open run_team1.py at the lines above; Cmd/Ctrl+Click a team function in VS Code opens the original file in code/.

run_team1.py step by step

- Adds code/ to the import path, turns off .pyc files so code/ stays clean, and copies everything printed into a log (the Tee class).
- Reads input/team1_inputs.json and calls random.seed(42).
- **Demo 1:** calls the team's main.main() unchanged, prints the registry, then clears it, resets the seed and repeats the first two registrations to show the numbers repeat.
- **Demo 2:** registers 'L12-4 PZT-5H' with (4000, 12000) kHz, looks it up in lower case, prints bandwidth and centre, and makes 2000 look-up calls.
- **Demo 3:** builds Piezo objects with five wrong ranges, sends one through process_material(), looks up an unknown name, and prints the exception family.
- Saves registry_contents.csv, two figures, console_log.txt and console_screenshot.png.

Full listing of run_team1.py

The numbers on the left are the real line numbers of the file, the same as in the tables above. A very long line continues on the next row (marked with an arrow) without a new number.

run_team1.py (runner written for this demo)

```

1  """
2  run_team1.py - one command to run Team 1 (Transducer) code and save all outputs.
3
4      macOS / Linux : python3 run_team1.py
5      Windows      : python run_team1.py
6
7  What it does
8      Demo 1  Runs the team's own main.py demo exactly as written.
9      Demo 2  Uses the team's functions for the project probe
10             (Philips L12-4 reference model, PZT-5H, 4 to 12 MHz = 4000 to 12000 kHz).
11      Demo 3  Sends wrong inputs to the team's code to show its validation errors.
12
13  The team's files in code/ are not changed. This script only imports and calls them.
14  All frequencies are in kHz, the unit used by the team's main.py.
15  Everything is saved in the output/ folder.
16  """
17  import csv
18  import io
19  import json
20  import random
21  import sys
22  import time
23  from pathlib import Path
24
25  HERE = Path(__file__).resolve().parent
26  CODE = HERE / "code"
27  INPUT = HERE / "input" / "team1_inputs.json"
28  OUT = HERE / "output"
29  OUT.mkdir(exist_ok=True)
30
31  # The team's modules import each other by plain name (e.g. "from registry import registry"),
32  # so the code/ folder must be on the import path.
33  sys.path.insert(0, str(CODE))
34  sys.dont_write_bytecode = True # keep code/ clean (no __pycache__ folder)
35
36  import matplotlib # noqa: E402
37
38  matplotlib.use("Agg")
39  import matplotlib.pyplot as plt # noqa: E402
40
41
```

```

42 class Tee(io.TextIOBase):
43     """Prints to the screen and keeps a copy of the text for the log file."""
44
45     def __init__(self, screen):
46         self.screen, self.buf = screen, io.StringIO()
47
48     def write(self, s):
49         self.screen.write(s)
50         self.buf.write(s)
51         return len(s)
52
53     def flush(self):
54         self.screen.flush()
55
56
57 def banner(text):
58     print("\n" + "=" * 72)
59     print(text)
60     print("=" * 72)
61
62
63 def console_to_png(text, out_png, title="Terminal", width_chars=106, max_lines=120):
64     """Save the console text as a terminal-style picture."""
65     lines = []
66     for line in text.rstrip("\n").expandtabs(4).splitlines():
67         while len(line) > width_chars:
68             lines.append(line[:width_chars])
69             line = " " + line[width_chars:]
70             lines.append(line)
71     if len(lines) > max_lines:
72         lines = lines[:max_lines - 1] + [f"... ({len(lines) - max_lines + 1} more lines, see conso
↳ le_log.txt)"]
73     h = 0.62 + 0.165 * len(lines)
74     fig = plt.figure(figsize=(0.083 * width_chars + 0.4, h))
75     fig.patch.set_facecolor("#1e1e1e")
76     fig.text(0.012, 1 - 0.28 / h, "o o o " + title, color="#9a9a9a", fontsize=9, family="DejaVu
↳ Sans Mono", va="center")
77     for i, ln in enumerate(lines):
78         fig.text(0.012, 1 - (0.55 + 0.165 * i) / h, ln, color="#e6e6e6", fontsize=8.6,
79                 family="DejaVu Sans Mono", va="top")
80     fig.savefig(out_png, dpi=160, facecolor=fig.get_facecolor())
81     plt.close(fig)
82
83
84 def main():
85     t0 = time.perf_counter()
86     cfg = json.loads(INPUT.read_text())
87     seed = cfg["random_seed"]
88
89     # The team's process_material() uses random.uniform() with no seed.
90     # We fix the seed here so that every run prints the same numbers.
91     random.seed(seed)
92
93     # Import the team's original modules (unchanged copies in code/).
94     import main as team_main
95     from registry import registry
96     from process import process_material
97     from piezo import Piezo
98     from exceptions import PiezoError, PiezoValidationError, PiezoNotFoundError
99
100     print("NITK-UsoundSim | Team 1: Transducer (piezoelectric material registry)")
101     print(f"Team code folder : code/ (original files, not modified)")
102     print(f"Input file       : input/team1_inputs.json")
103     print(f"Random seed      : {seed} (set by run_team1.py so results repeat)")
104     print(f"Frequency unit    : kHz (1 MHz = 1000 kHz), same as the team's main.py")
105
106     # -----
107     banner("DEMO 1: the team's own main.py (run exactly as written)")
108     team_main.main()
109     print("\nRegistry after Demo 1:")
110     for name in registry.list_materials():
111         lo, hi = registry.get_material(name).freq_range
112         print(f" {name:<10s} {lo:9.1f} to {hi:9.1f} kHz")
113     print("Note: main.py's 'BadRange' test uses (500, 1000), which is a VALID range,")
114     print("       so no validation error is raised there and 'BadRange' is registered.")
115     print("       Demo 3 below shows the validation errors with really invalid ranges.")
116

```

```

117     demol_freqs = {}
118     # Re-create the numbers main.py printed, to mark them on the figure.
119     random.seed(seed)
120     registry.clear()
121     demol_freqs["Quartz"] = process_material("Quartz", (100.0, 500.0), create_new=True)
122     demol_freqs["PZT-5H"] = process_material("PZT-5H", (1000.0, 5000.0), create_new=True)
123     print("\n(Check) registry.clear(), seed reset, Quartz and PZT-5H registered again:")
124     print(f"      Quartz {demol_freqs['Quartz']:.2f} kHz and PZT-5H {demol_freqs['PZT-5H']:.2f}
    ↪ kHz,"
125           " the same as above, so the run is repeatable.")
126
127     # -----
128     banner("DEMO 2: the project probe (Philips L12-4 reference model)")
129     p = cfg["demo_2_project_probe"]
130     lo_in, hi_in = p["freq_range_khz"]
131     print(f"Probe      : {p['probe_name']}")
132     print(f"Material      : {p['material']}")
133     print(f"Elements       : {p['n_elements']} (element positions are made elsewhere, by Team 3
    ↪ 's code)")
134     print(f"Operating range : {lo_in/1000:.0f} to {hi_in/1000:.0f} MHz = {lo_in:.0f} to {hi_in:.0
    ↪ f} kHz")
135
136     f_probe = process_material(p["name"], (lo_in, hi_in), create_new=True)
137     print(f"\nprocess_material('{p['name']}', ({lo_in:.0f}, {hi_in:.0f}), create_new=True)")
138     print(f"  -> random frequency inside the range: {f_probe:.2f} kHz = {f_probe/1000:.3f} MHz")
139
140     obj = registry.get_material(p["name"].lower()) # lookup is case-insensitive
141     print(f"registry.get_material('{p['name'].lower()}') -> Piezo(name='{obj.name}', freq_range={o
    ↪ bj.freq_range}")
142     bw = obj.freq_range[1] - obj.freq_range[0]
143     centre = (obj.freq_range[0] + obj.freq_range[1]) / 2
144     print(f"  bandwidth = max - min = {bw:.0f} kHz, middle of range = {centre:.0f} kHz "
145           f"(project centre frequency {p['center_frequency_khz']:.0f} kHz)")
146     print(f"  fractional bandwidth = bandwidth / middle = {bw/centre*100:.0f} %")
147
148     n = p["n_random_draws"]
149     draws = [process_material(p["name"], create_new=False) for _ in range(n)]
150     inside = sum(lo_in <= f <= hi_in for f in draws)
151     print(f"\n{n} more calls with create_new=False (look up only):")
152     print(f"  smallest = {min(draws):.2f} kHz, largest = {max(draws):.2f} kHz, "
153           f"average = {sum(draws)/n:.2f} kHz")
154     print(f"  values inside {lo_in:.0f}..{hi_in:.0f} kHz: {inside} of {n}")
155     print(f"Registered materials now: {registry.list_materials()}")
156
157     # Save registry contents (valid materials only) for the figure and CSV.
158     reg_rows = []
159     for name in registry.list_materials():
160         m = registry.get_material(name)
161         gen = demol_freqs.get(name, f_probe if name == p["name"] else None)
162         reg_rows.append((name, m.freq_range[0], m.freq_range[1], gen))
163
164     # -----
165     banner("DEMO 3: validation errors (wrong inputs)")
166     print("3a) Making a Piezo object directly with a bad range:")
167     for case in cfg["demo_3_invalid_inputs"]:
168         rng = case["freq_range"]
169         rng = tuple(rng) if isinstance(rng, list) else rng
170         try:
171             Piezo(case["name"], rng)
172             print(f"  {case['name']:<12s} {rng!r}: accepted (no error)")
173         except PiezoValidationError as e:
174             print(f"  {case['name']:<12s} {rng!r}")
175             print(f"    problem : {case['problem']}")
176             print(f"    caught  : PiezoValidationError: {e}")
177
178     print("\n3b) The same kind of bad input through process_material():")
179     try:
180         process_material("Swapped", (12000.0, 4000.0), create_new=True)
181     except PiezoValidationError as e:
182         print(f"  caught PiezoValidationError: {e}")
183     left = registry.get_material("Swapped")
184     print(f"  but 'Swapped' is still in the registry with freq_range={left.freq_range}")
185     print(f"  (newPiezoMaterial() adds it with (0.0, 0.0) BEFORE the range is checked)")
186
187     print("\n3c) Looking up a material that was never registered:")
188     try:
189         registry.get_material("PZT-8")

```

```

190     except PiezoNotFoundError as e:
191         print(f" caught PiezoNotFoundError: {e}")
192
193     print("\n3d) Exception family (from exceptions.py):")
194     print(f" PiezoValidationError is a PiezoError: {issubclass(PiezoValidationError, PiezoError)}")
195     print(f" PiezoNotFoundError is a PiezoError: {issubclass(PiezoNotFoundError, PiezoError)}")
196     print(f" PiezoNotFoundError is a ValueError: {issubclass(PiezoNotFoundError, ValueError)}")
197     print(f" PiezoNotFoundError is a KeyError : {issubclass(PiezoNotFoundError, KeyError)}")
198
199     # -----
200     banner("SAVING OUTPUTS")
201     csv_path = OUT / "registry_contents.csv"
202     with open(csv_path, "w", newline="") as fh:
203         w = csv.writer(fh)
204         w.writerow(["material", "min_freq_khz", "max_freq_khz", "bandwidth_khz", "generated_freq_khz"])
205         for name, lo, hi, gen in reg_rows:
206             w.writerow([name, lo, hi, hi - lo, "" if gen is None else round(gen, 2)])
207         print(f"saved {csv_path.relative_to(HERE)}")
208
209     # Figure 1: frequency ranges of valid registered materials.
210     fig, ax = plt.subplots(figsize=(9, 3.8))
211     colours = {"Quartz": "#8c8c8c", "PZT-5H": "#2e6da4", "BadRange": "#c0a060", p["name"]: "#b0413e"}
212     for i, (name, lo, hi, gen) in enumerate(reg_rows):
213         ax.barh(i, hi - lo, left=lo, height=0.5, color=colours.get(name, "#555"), alpha=0.85)
214         ax.text(hi * 1.06, i, f"{lo:g} to {hi:g} kHz", va="center", fontsize=9)
215         if gen is not None:
216             ax.plot(gen, i, marker="v", color="black", markersize=10)
217             ax.text(gen, i + 0.33, f"{gen:.1f}", ha="center", fontsize=8)
218     ax.set_yticks(range(len(reg_rows)), [r[0] for r in reg_rows])
219     ax.set_xscale("log")
220     ax.set_xlim(60, 40000)
221     ax.set_ylim(-0.6, len(reg_rows) - 0.3)
222     ax.set_xlabel("Frequency (kHz, log scale)")
223     ax.set_title(f"Registered piezo materials: frequency range (bar) and random frequency from process_material() (triangle), seed {seed}",
224                 fontsize=9.5)
225     ax.grid(axis="x", which="both", alpha=0.3)
226     fig.tight_layout()
227     f1 = OUT / "fig1_material_frequency_ranges.png"
228     fig.savefig(f1, dpi=150)
229     plt.close(fig)
230     print(f"saved {f1.relative_to(HERE)}")
231
232     # Figure 2: histogram of the random frequencies for the project probe.
233     fig, ax = plt.subplots(figsize=(8, 3.8))
234     ax.hist([d / 1000 for d in draws], bins=40, range=(3, 13), color="#b0413e", alpha=0.8, edgecolor="white")
235     ax.axvline(lo_in / 1000, color="black", ls="--", lw=1)
236     ax.axvline(hi_in / 1000, color="black", ls="--", lw=1)
237     ax.axvline(p["center_frequency_khz"] / 1000, color="#2e6da4", lw=1.5)
238     ymax = ax.get_ylim()[1] * 1.25
239     ax.set_ylim(0, ymax)
240     ax.text(lo_in / 1000, ymax * 0.95, " min 4 MHz", fontsize=8, va="top")
241     ax.text(hi_in / 1000, ymax * 0.95, " max 12 MHz", fontsize=8, va="top")
242     ax.text(p["center_frequency_khz"] / 1000, ymax * 0.85, " centre 8 MHz", fontsize=8, va="top",
243           color="#2e6da4")
244     ax.set_xlabel("Frequency returned by process_material() (MHz)")
245     ax.set_ylabel("Count")
246     ax.set_title(f"L12-4 PZT-5H: {n} calls to process_material(create_new=False), seed {seed}",
247                 fontsize=10)
248     fig.tight_layout()
249     f2 = OUT / "fig2_l12_4_random_frequencies.png"
250     fig.savefig(f2, dpi=150)
251     plt.close(fig)
252     print(f"saved {f2.relative_to(HERE)}")
253
254     print(f"saved output/console_log.txt")
255     print(f"saved output/console_screenshot.png")
256     print(f"\nDone in {time.perf_counter() - t0:.1f} s. All outputs are in the output/ folder.")
257
258 if __name__ == "__main__":
259     tee = Tee(sys.stdout)

```

```
259     sys.stdout = tee
260     try:
261         main()
262     finally:
263         sys.stdout = tee.screen
264     text = tee.buf.getvalue()
265     (OUT / "console_log.txt").write_text(text)
266     console_to_png(text, OUT / "console_screenshot.png", title="python3 run_team1.py")
```

5 Input

What goes into the run

The team's code takes simple parameters: a material name and a (min, max) frequency pair. The runner reads them from `input/team1_inputs.json`. All frequencies are in kHz.

Demo 1: values written inside main.py

Material	Range (kHz)	Note
Quartz	100 to 500	
PZT-5H	1000 to 5000	
BadRange	500 to 1000	meant as a bad range, but it is valid

Demo 2: the project probe

Item	Value
Registry name	L12-4 PZT-5H
Probe	Philips L12-4 reference model
Material	PZT-5H
Elements	128
Frequency range	4000 to 12000 kHz (4 to 12 MHz)
Centre frequency (project)	8000 kHz (8 MHz)
Extra look-up calls	2000
Random seed	42

These probe values come from the project's probe file (material PZT-5H, 128 elements, 4 to 12 MHz).

Demo 3: wrong inputs

Name	freq_range given	Problem
Swapped	[12000.0, 4000.0]	minimum is larger than maximum
Negative	[-500.0, 4000.0]	negative frequency
Text	["four MHz", 12000.0]	value is not a number
ThreeValues	[4000.0, 8000.0, 12000.0]	three values instead of two
NotATuple	8000.0	a single number instead of a (min, max) pair

6 Output

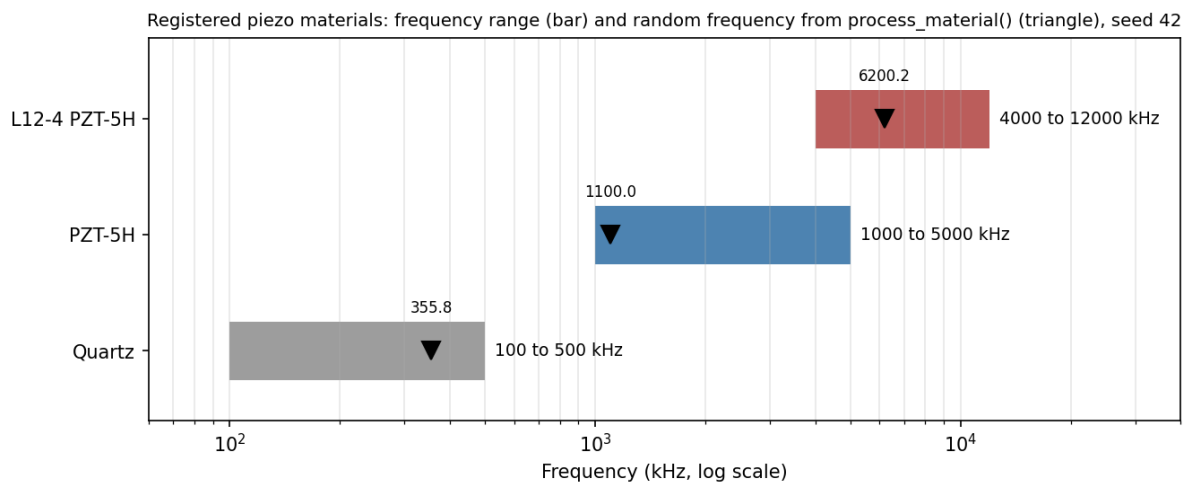
Real results of `python3 run_team1.py` (seed 42)

Registry contents

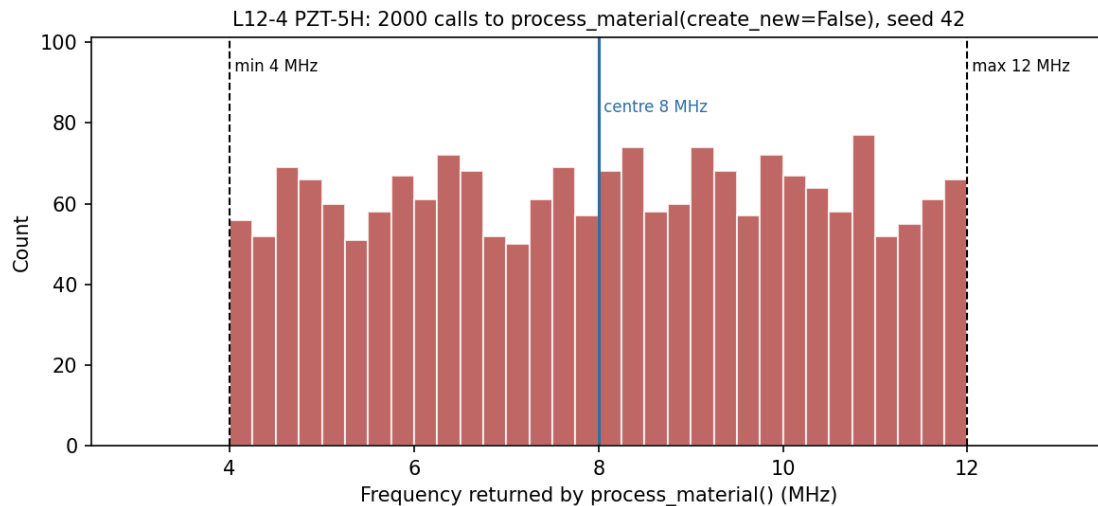
Material	Min (kHz)	Max (kHz)	Bandwidth (kHz)	Generated (kHz)
Quartz	100.0	500.0	400.0	355.77
PZT-5H	1000.0	5000.0	4000.0	1100.04
L12-4 PZT-5H	4000.0	12000.0	8000.0	6200.23

From `output/registry_contents.csv`. The table is taken after the registry was cleared and rebuilt (so 'BadRange' from `main.py` is not in it) and before Demo 3.

Figure 1: material frequency ranges



Each bar is the (min, max) range stored in the registry, on a log scale. The triangle and number show the random frequency returned by `process_material()`: 355.8 kHz for Quartz, 1100.0 kHz for PZT-5H (main.py), 6200.2 kHz for the L12-4 probe. Each triangle lies inside its bar, as it must.

Figure 2: random frequencies for the L12-4 probe

Histogram of 2000 values from process_material('L12-4 PZT-5H', create_new=False). The bars are roughly equal height between 4 and 12 MHz and nothing lies outside: random.uniform gives a flat (uniform) distribution. The average was 8046.37 kHz, close to the 8 MHz centre.

Console output

Full text printed by the run (also saved as output/console_log.txt). The last line shows the time measured on that run.

python3 run_team1.py

```
NITK-UsoundSim | Team 1: Transducer (piezoelectric material registry)
Team code folder : code/ (original files, not modified)
Input file       : input/team1_inputs.json
Random seed      : 42 (set by run_team1.py so results repeat)
Frequency unit   : kHz (1 MHz = 1000 kHz), same as the team's main.py

=====
DEMO 1: the team's own main.py (run exactly as written)
=====
--- 1. Registering Materials ---
Quartz generated frequency : 355.77 kHz
PZT-5H generated frequency: 1100.04 kHz
Registered materials       : ['Quartz', 'PZT-5H']

--- 2. Retrieving Existing Material ---
Retrieved Quartz frequency : 210.01 kHz

--- 3. Testing Exception Handling ---
Lookup failure caught      : "Material 'UnknownMat' was not found in the registry."

Registry after Demo 1:
Quartz      100.0 to 500.0 kHz
PZT-5H      1000.0 to 5000.0 kHz
BadRange    500.0 to 1000.0 kHz
Note: main.py's 'BadRange' test uses (500, 1000), which is a VALID range,
so no validation error is raised there and 'BadRange' is registered.
Demo 3 below shows the validation errors with really invalid ranges.

(Check) registry.clear(), seed reset, Quartz and PZT-5H registered again:
Quartz 355.77 kHz and PZT-5H 1100.04 kHz, the same as above, so the run is repeatable.

=====
DEMO 2: the project probe (Philips L12-4 reference model)
=====
Probe       : Philips L12-4 reference model
Material    : PZT-5H
Elements    : 128 (element positions are made elsewhere, by Team 3's code)
Operating range : 4 to 12 MHz = 4000 to 12000 kHz
```

```

process_material('L12-4 PZT-5H', (4000, 12000), create_new=True)
-> random frequency inside the range: 6200.23 kHz = 6.200 MHz
registry.get_material('l12-4 pzt-5h') -> Piezo(name='L12-4 PZT-5H', freq_range=(4000.0, 12000.0))
bandwidth = max - min = 8000 kHz, middle of range = 8000 kHz (project centre frequency 8000 kHz)
fractional bandwidth = bandwidth / middle = 100 %

2000 more calls with create_new=False (look up only):
smallest = 4003.25 kHz, largest = 11999.26 kHz, average = 8046.37 kHz
values inside 4000..12000 kHz: 2000 of 2000
Registered materials now: ['Quartz', 'PZT-5H', 'L12-4 PZT-5H']

=====
DEMO 3: validation errors (wrong inputs)
=====

3a) Making a Piezo object directly with a bad range:
Swapped      (12000.0, 4000.0)
  problem : minimum is larger than maximum
  caught  : PiezoValidationError: Minimum frequency (12000.0) cannot exceed maximum frequency (4000.
↳ 0).
Negative      (-500.0, 4000.0)
  problem : negative frequency
  caught  : PiezoValidationError: Frequencies must be non-negative. Got: (-500.0, 4000.0)
Text          ('four MHz', 12000.0)
  problem : value is not a number
  caught  : PiezoValidationError: Frequency values must be numeric. Details: could not convert strin
↳ g to float: 'four MHz'
ThreeValues   (4000.0, 8000.0, 12000.0)
  problem : three values instead of two
  caught  : PiezoValidationError: freq_range must be a tuple or list of 2 numbers (min_freq, max_fre
↳ q).
NotATuple     8000.0
  problem : a single number instead of a (min, max) pair
  caught  : PiezoValidationError: freq_range must be a tuple or list of 2 numbers (min_freq, max_fre
↳ q).

3b) The same kind of bad input through process_material():
caught PiezoValidationError: Minimum frequency (12000.0) cannot exceed maximum frequency (4000.0).
but 'Swapped' is still in the registry with freq_range=(0.0, 0.0)
(newPiezoMaterial() adds it with (0.0, 0.0) BEFORE the range is checked)

3c) Looking up a material that was never registered:
caught PiezoNotFoundError: "Material 'PZT-8' was not found in the registry."

3d) Exception family (from exceptions.py):
PiezoValidationError is a PiezoError: True, is a ValueError: True
PiezoNotFoundError is a PiezoError: True, is a KeyError : True

=====
SAVING OUTPUTS
=====
saved output/registry_contents.csv
saved output/fig1_material_frequency_ranges.png
saved output/fig2_l12_4_random_frequencies.png
saved output/console_log.txt
saved output/console_screenshot.png

Done in 0.4 s. All outputs are in the output/ folder.

```

What the console shows

Demo 1: main.py works: Quartz 355.77 kHz, PZT-5H 1100.04 kHz, Quartz again 210.01 kHz, and the UnknownMat lookup error.

Demo 1 note: main.py's 'BadRange' check does not trigger, because (500, 1000) is valid.

Demo 2: the L12-4 range is stored, found with a lower-case name, and all 2000 random values are inside 4000..12000 kHz.

Demo 3: every wrong input gives a PiezoValidationError with a clear message. Through process_material(), the bad name stays in the registry with (0.0, 0.0).

Console screenshot

```

o o o  python3 run_team1.py

NITK-UsoundSim | Team 1: Transducer (piezoelectric material registry)
Team code folder : code/ (original files, not modified)
Input file       : input/team1_inputs.json
Random seed      : 42 (set by run_team1.py so results repeat)
Frequency unit   : kHz (1 MHz = 1000 kHz), same as the team's main.py

=====
DEMO 1: the team's own main.py (run exactly as written)
=====
--- 1. Registering Materials ---
Quartz generated frequency : 355.77 kHz
PZT-5H generated frequency : 1100.04 kHz
Registered materials       : ['Quartz', 'PZT-5H']

--- 2. Retrieving Existing Material ---
Retrieved Quartz frequency : 210.01 kHz

--- 3. Testing Exception Handling ---
Lookup failure caught      : "Material 'UnknownMat' was not found in the registry."

Registry after Demo 1:
Quartz      100.0 to 500.0 kHz
PZT-5H      1000.0 to 5000.0 kHz
BadRange    500.0 to 1000.0 kHz
Note: main.py's 'BadRange' test uses (500, 1000), which is a VALID range,
      so no validation error is raised there and 'BadRange' is registered.
      Demo 3 below shows the validation errors with really invalid ranges.

(Check) registry.clear(), seed reset, Quartz and PZT-5H registered again:
      Quartz 355.77 kHz and PZT-5H 1100.04 kHz, the same as above, so the run is repeatable.

=====
DEMO 2: the project probe (Philips L12-4 reference model)
=====
Probe       : Philips L12-4 reference model
Material    : PZT-5H
Elements    : 128 (element positions are made elsewhere, by Team 3's code)
Operating range : 4 to 12 MHz = 4000 to 12000 kHz

process_material('L12-4 PZT-5H', (4000, 12000), create_new=True)
-> random frequency inside the range: 6200.23 kHz = 6.200 MHz
registry.get_material('l12-4 pzt-5h') -> Piezo(name='L12-4 PZT-5H', freq_range=(4000.0, 12000.0))
bandwidth = max - min = 8000 kHz, middle of range = 8000 kHz (project centre frequency 8000 kHz)
fractional bandwidth = bandwidth / middle = 100 %

2000 more calls with create_new=False (look up only):
  smallest = 4003.25 kHz, largest = 11999.26 kHz, average = 8046.37 kHz
  values inside 4000..12000 kHz: 2000 of 2000
Registered materials now: ['Quartz', 'PZT-5H', 'L12-4 PZT-5H']

=====
DEMO 3: validation errors (wrong inputs)
=====
3a) Making a Piezo object directly with a bad range:
  Swapped      (12000.0, 4000.0)
    problem : minimum is larger than maximum
    caught  : PiezoValidationError: Minimum frequency (12000.0) cannot exceed maximum frequency (4000.0)
  .
  Negative     (-500.0, 4000.0)
    problem : negative frequency
    caught  : PiezoValidationError: Frequencies must be non-negative. Got: (-500.0, 4000.0)
  Text         ('four MHz', 12000.0)
    problem : value is not a number
    caught  : PiezoValidationError: Frequency values must be numeric. Details: could not convert string
              to float: 'four MHz'
  ThreeValues  (4000.0, 8000.0, 12000.0)
    problem : three values instead of two
    caught  : PiezoValidationError: freq_range must be a tuple or list of 2 numbers (min_freq, max_freq)
  .
  NotATuple    8000.0
    problem : a single number instead of a (min, max) pair
    caught  : PiezoValidationError: freq_range must be a tuple or list of 2 numbers (min_freq, max_freq)
  .

3b) The same kind of bad input through process_material():
  caught PiezoValidationError: Minimum frequency (12000.0) cannot exceed maximum frequency (4000.0).
  but 'Swapped' is still in the registry with freq_range=(0.0, 0.0)
  (newPiezoMaterial() adds it with (0.0, 0.0) BEFORE the range is checked)

3c) Looking up a material that was never registered:
  caught PiezoNotFoundError: "Material 'PZT-8' was not found in the registry."

3d) Exception family (from exceptions.py):
  PiezoValidationError is a PiezoError: True, is a ValueError: True
  PiezoNotFoundError   is a PiezoError: True, is a KeyError   : True

=====
SAVING OUTPUTS
=====
saved output/registry_contents.csv
saved output/fig1_material_frequency_ranges.png
saved output/fig2_l12_4_random_frequencies.png
saved output/console_log.txt
saved output/console_screenshot.png

Done in 0.4 s. All outputs are in the output/ folder.

```

output/console_screenshot.png: picture of the same console output.

7 How to run

Step by step

1. Install **Python 3.9 or newer** from python.org. On Windows tick 'Add Python to PATH'. (Tested with Python 3.12.)
2. Open a terminal in the **Team_1_Transducer** folder. macOS: right-click the folder in Finder → 'New Terminal at Folder'. Windows: open the folder, type cmd in the address bar, press Enter.
3. Install the package: macOS/Linux `python3 -m pip install -r requirements.txt`; Windows `python -m pip install -r requirements.txt`.
4. Run: macOS/Linux `python3 run_team1.py`; Windows `python run_team1.py`.
5. Read the console, then open the **output/** folder.

Expected runtime: about 1 second (measured 1.1 s wall time on a MacBook).

Outputs: output/console_log.txt, output/console_screenshot.png, output/fig1_material_frequency_ranges.png, output/fig2_l12_4_random_frequencies.png, output/registry_contents.csv.

Troubleshooting

Problem	Fix
python3: command not found (Windows)	Use python instead of python3.
No module named 'matplotlib'	Run the pip install step again.
TypeError: 'type' object is not subscriptable	Python is older than 3.9; install a newer version.
No module named 'registry'	Run from inside the folder and keep code/ next to run_team1.py.
Numbers are different	Check random_seed is 42 in input/team1_inputs.json.

8 Check yourself

Viva-style questions

Why can the same element send and receive ultrasound?

The piezoelectric effect works both ways: voltage makes it vibrate (transmit), and a returning pressure wave makes a voltage (receive).

Why do medical probes use PZT and not quartz?

PZT converts electrical to mechanical energy much more efficiently, so echoes are stronger, and it is easy to cut into many small array elements.

What is the bandwidth and centre frequency of the L12-4 range in this project?

$B = 12 - 4 = 8$ MHz, centre = 8 MHz, fractional bandwidth 100 % (the run prints 8000 kHz and 100 %).

Where is the range checked, and what four checks are made?

In the `freq_range` setter of Piezo (`piezo.py`): two values, both numeric, none negative, $\min \leq \max$. It runs on every assignment, including in `__init__`.

Why does 'BadRange' in `main.py` not print an error?

Its range (500, 1000) is valid, so no `PiezoValidationError` is raised; it is simply registered.

What happens to the registry when `process_material()` gets an invalid range?

`newPiezoMaterial()` has already added the name with (0.0, 0.0), then the update fails. The error is raised but the name stays in the registry with (0.0, 0.0).